CIFP Francesc de Borja Moll · C/ Caracas 6, 07007 Palma
cifpfrancescdeborjamoll@educaib.eu · 971 278 150

VinylEth

Final Project Report

Intermodular Project — UD3: Final Submission and Defence

Student: Stepan Andreev

Course: Projecte Intermodular UD3

Date: June 2026

Repository: github.com/Castor909/projecte-intermodular

Full-stack web application for an online vinyl record store
with Ethereum/MetaMask payment integration — MERN Stack

Contents

1. Introduction

VinylEth is a full-stack web application for an online vinyl record store with integrated cryptocurrency payments. The project targets a real market niche: vinyl has experienced a significant commercial revival over the last fifteen years, and a segment of its buyers are also participants in the Web3 ecosystem. VinylEth serves vinyl enthusiasts who value privacy and decentralised finance by letting them browse a curated catalog and complete purchases directly through MetaMask, without relying on traditional payment processors.

The application is built on the MERN stack (MongoDB, Express, React, Node.js) and integrates Ethereum's Sepolia testnet for payment transactions. The store side covers a complete e-commerce flow: browsing, search and filtering, a persistent cart, a wishlist, user accounts, a shipping step, and checkout with receipt emails. The admin side provides a complete product management interface including direct Discogs API imports and order management.

The project was developed over a full academic year across four successive phases:

- **UD1A** — Initial project setup, design system, and static prototype.
- **UD1B** — Backend integration: MongoDB, REST API, frontend connected to real data, cart.
- **UD2A** — Wallet UX, cart depth, catalog search and filtering, API error contracts.
- **UD3** — Payment flow, authentication, admin panel, email receipts, audio player, wishlist, and full polish.

2. Objectives

2.1 Initial Objectives

At the start of the project, the following goals were defined:

- Build a functional MERN e-commerce application as a foundation.
- Implement a product catalog connected to a real database.
- Build a persistent shopping cart.
- Integrate Ethereum payments via MetaMask as the distinguishing technical feature.
- Maintain professional-quality documentation throughout the course.

2.2 Final State vs. Initial Goals

All initial objectives were met and significantly exceeded. The final application includes areas that were not planned at the outset, including a complete admin panel with external API integrations, a wishlist system, a custom audio preview player, PWA support, and an order history module.

tablegray **Objective**	**Planned**	**Delivered**
Catalog from database	Yes	Yes
Shopping cart + persistence	Yes	Yes
MetaMask payment flow	Yes	Yes
User authentication	No	Yes — full JWT auth
Admin panel	No	Yes — full CRUD + stats
Email receipts	No	Yes — via Resend API
Audio previews	No	Yes — via iTunes API
Wishlist	No	Yes
Order history	No	Yes
PWA support	No	Yes

3. Technologies Used

3.1 Core Stack

tablegray **Technology**	**Version**	**Role**
React	19	Frontend UI library (SPA)
React Router	7	Client-side routing
Vite	7	Frontend build tool
Node.js	20.19+	JavaScript runtime
Express	5	Backend HTTP framework
MongoDB	7	NoSQL document database
Mongoose	9	MongoDB ODM

3.2 Authentication and Security

- **JWT (jsonwebtoken)** — stateless session tokens with 7-day expiry, verified by a custom `requireAuth` middleware on protected routes.
- **bcryptjs** — passwords are hashed before storage; plaintext passwords are never persisted.
- A separate **admin token middleware** (`requireAdmin.js`) guards admin-only endpoints using a distinct secret, decoupled from the user JWT system.

3.3 Payments and Web3

MetaMask is used as the browser wallet for transaction signing. No additional Web3 library was added; the native `window.ethereum` provider API (`eth_requestAccounts`, `eth_sendTransactio` was used directly to keep dependencies minimal and reduce third-party surface area.

All ETH amounts are converted to Wei using JavaScript `BigInt` arithmetic, splitting the decimal representation at the decimal point and scaling each part independently. This avoids the floating-point precision errors that occur with standard IEEE 754 arithmetic at small ETH values.

Payments target the Ethereum **Sepolia testnet** (chain ID `0xaa36a7`).

3.4 External API Integrations

- **Discogs API** — fetches album metadata (title, artist, year, genre, label, country, barcode, full tracklist with durations, and cover image URL) by release ID. Used in the admin panel's Quick Import feature to populate the album form in one step.
- **iTunes Search API** — fetches a 30-second audio preview URL for a given artist and title. Used to auto-fill the audio preview field when creating or editing an album.

3.5 Email

Resend is used for transactional email. After a successful MetaMask payment, the server sends an order receipt to the customer's email address. On the Resend free plan, the recipient is redirected to a configured fallback address.

3.6 Additional Tools

- **dotenv** — environment variable management; secrets are never hardcoded.
- **CORS** — enabled on the Express server for the configured `CLIENT_URL`.
- **nodemon** — development server auto-restart on file changes.
- **PWA Manifest** — `public/manifest.json` with a custom SVG icon enables Add to Home Screen on mobile browsers.

4. Architecture and Structure

4.1 Overview

The project follows a standard client-server separation. The frontend is a React SPA served by Vite during development (port 5173). The backend is a REST API served by Express (port 5000). They communicate over HTTP; the frontend never accesses the database directly.

```
projecte-intermodular/├──
 client/                  # React SPA (Vite)|├──
    src/|||├──
       pages/           # Full-page route components|||├──
       components/      # Reusable UI components|||├──
       api/             # Fetch wrapper functions|||├──
       services/        # Business logic (cart state, price utils)|||├──
       context/         # React Contexts (Auth, Cart, Wishlist)|||└──
       config/          # App-level constants (store wallet address)|└──
    public/              # Static assets, PWA manifest└──
 server/                  # Express REST API├──
     routes/              # Route definitions and handlers├──
     controllers/         # Business logic (albums)├──
     models/              # Mongoose schemas (Album, User, Order)├──
     middleware/          # JWT and admin token guards├──
     utils/               # Helpers (Discogs mapper, email, price)├──
     config/              # Database connection module└──
     seed/                # Initial dataset population script
```

4.2 Data Models

Album

`title`, `artist`, `year`, `genre`, `priceEth`, `coverUrl`, `stock`, `featured` (bool), `description`, `audioUrl`, `tracks[]{title, duration}`, `label`, `country`, `vinylFormat`, `barcode`, `mbid`, `discountPercent`

User

`email` (unique, lowercase), `passwordHash`, `savedAddress{fullName, address, city, postalCode, country}`

Order

`txHash`, `totalEth`, `items[]{albumId, title, artist, qty, priceEth}`, `shippingAddress{...}`, `userId` (optional — anonymous checkout is supported), `createdAt`

4.3 API Endpoints

Method	Endpoint	Auth	Description
GET	`/api/albums`	—	Paginated list (search, filter, sort)
GET	`/api/albums/genres`	—	Distinct genre list from DB
GET	`/api/albums/:id`	—	Single album detail
POST	`/api/albums`	Admin	Create album
PUT	`/api/albums/:id`	Admin	Update album
DELETE	`/api/albums/:id`	Admin	Delete album
POST	`/api/auth/register`	—	Register new user
POST	`/api/auth/login`	—	Login, returns JWT
PUT	`/api/auth/address`	JWT	Save delivery address
POST	`/api/auth/change-password`	JWT	Change user password
POST	`/api/orders`	—	Create order after payment
GET	`/api/orders/mine`	JWT	Current user's order history
POST	`/api/admin/login`	—	Admin authentication
GET	`/api/admin/stats`	Admin	Dashboard statistics
GET	`/api/admin/orders`	Admin	All orders (search + filter)
GET	`/api/discogs/:id`	Admin	Fetch release from Discogs
GET	`/api/itunes`	Admin	Fetch audio preview from iTunes

4.4 State Management

React Context is used for all global frontend state. Three contexts exist, each with a corresponding custom hook:

- **AuthContext / useAuth** — current user object, JWT token, login, logout, and register actions. The token is persisted to `localStorage` and rehydrated on page load.
- **CartContext / useCart** — cart items, quantities, totals, and all mutation actions (add, increment, decrement, remove, clear). The cart is persisted to `localStorage`.
- **WishlistContext / useWishlist** — wishlist items and toggle action. Persisted to `localStorage`.

No third-party state management library was used. The combination of Context and custom hooks is sufficient for the application's scale and avoids unnecessary dependencies.

5. Implemented Functionality

5.1 Catalog and Product Browsing

The catalog page loads albums from the backend API with server-side pagination via a "Load more" button. Users can search across title, artist, and genre using a debounced input (250 ms delay) to avoid excessive requests during fast typing. A genre dropdown fetches the available genres from the database at runtime. Albums can be sorted by title, price, year, stock quantity, and featured status — eight sort options in total.

Each album card shows the cover image, title, artist, price in ETH, a stock status badge ("Out of Stock" or "Last Copies" when stock is at or below 2), and a wishlist toggle. Opening a card navigates to a detail page with full metadata, a tracklist with individual track durations, a 30-second audio preview player when an iTunes URL is available, a similar albums section (same genre, four items), and an add-to-cart button.

The homepage shows a featured album section (albums with `featured: true`) and a special offers section (albums with a non-zero `discountPercent`). A recently viewed shelf tracks the last six albums opened, stored in `localStorage`. Skeleton loading cards are shown during data fetching to prevent layout shifts.

5.2 Shopping Cart

Cart state is managed globally through `CartContext`. Items can be added from both the catalog card and the detail page. Stock limits are enforced client-side: quantity cannot exceed available stock, and adding an out-of-stock album is blocked with a visible warning. The quantity can be incremented or decremented per line; decrementing at 1 removes the item entirely. Prices respect any active discount through a shared `effectivePrice()` utility. The full cart state persists across page refreshes via `localStorage`.

5.3 Wishlist

Any album can be toggled into or out of the wishlist from both the catalog card and the detail page. A badge in the header shows the wishlist count when non-zero. The wishlist page renders the saved albums as full cards with the same add-to-cart action available. Wishlist state persists to `localStorage` under the key `vinyleth_wishlist`.

5.4 Checkout Flow

Checkout is a three-step linear flow: **Cart** → **Shipping** → **Payment**.

The shipping step presents a validated form requiring full name, address, city, postal code, and country. If the logged-in user has a saved address on their profile, the form is pre-filled automatically. Submitting the form saves the address to the user's profile via `PUT /api/auth/address` and navigates to the payment step.

The payment step shows an order summary (items, quantities, individual prices, and total in ETH) followed by a "Pay with MetaMask" button. On activation:

1. The app requests a chain switch to Sepolia (`0xaa36a7`) if the wallet is on a different network.
2. `eth_sendTransaction` is called with the `STORE_WALLET` address and the total converted to Wei via `BigInt`.
3. On confirmation, the transaction hash is displayed with a direct Etherscan Sepolia link.
4. A `POST /api/orders` request saves the order to the database (items, shipping address, TX hash, total ETH).
5. A receipt email is dispatched via Resend.
6. The cart is cleared.

5.5 User Authentication

Users register with an email address and a password of at least six characters. Duplicate email addresses are rejected at the database level via a unique index. Passwords are hashed with bcrypt before storage. Login returns a JWT with a 7-day expiry stored in `localStorage` and attached as a `Bearer` token on subsequent authenticated requests.

The router protects pages that require a logged-in session; unauthenticated users are redirected to `/login`. The profile page allows viewing the account email, changing the

password (which requires the current password), and managing the saved delivery address. An order history page lists all past orders with their items, totals in ETH, TX hashes, and Etherscan links.

5.6 Admin Panel

The admin panel is accessible at `/admin` and uses a separate password-only login, independent of the user account system. The session token is stored under a different `localStorage` key and sent as an `x-admin-token` request header.

The dashboard displays live statistics: total album count, number of out-of-stock albums, total orders placed, and total revenue in ETH, alongside a top-5 best-selling albums list.

The album management tab supports creating and editing albums through a full form. Entering a Discogs release ID auto-fills the form with title, artist, year, genre, label, country, barcode, complete tracklist, and cover image URL from the Discogs API. The audio preview field can be auto-filled from the iTunes Search API. Individual albums can be deleted. Batch operations allow selecting multiple albums to set a discount percentage, toggle featured status, or delete them in one action. The current filtered album list can be exported to a dated CSV file.

The orders tab lists all placed orders with buyer email (or “Anonymous” for guest checkouts), items purchased, total ETH, transaction hash, and date. The list supports free-text search by TX hash substring and filtering by a custom date range.

5.7 Audio Player

Albums that have an `audioUrl` field display a custom HTML5 audio player on their detail page. The player provides play/pause control, a seekable progress bar showing elapsed and total time, and a buffering indicator for slow connections. The component manages its own state independently of the global contexts.

5.8 Progressive Web App

A `manifest.json` in the client’s `public` directory defines the application name, theme and background colours, `standalone` display mode, and a custom SVG icon. This enables Add to Home Screen prompts on supported mobile browsers and allows the app to launch without browser chrome when installed.

6. Problems Encountered and Solutions

6.1 MongoDB crash on the development machine

During the UD2A phase, the system's MongoDB package (`mongodb-bin` 8.2.7 on an Arch-based Linux) crashed with a `SIGSEGV` signal at startup, making the backend unable to connect to the database.

Solution: A containerised MongoDB 7 instance was started via Docker (`docker run -d --name mongo -p 27017:27017 mongo:7`). This provided a stable database for the rest of development without requiring any change to application code, since the connection string `mongodb://127.0.0.1:27017/vinyleth` resolved to the container correctly.

6.2 React fast-refresh lint error on CartContext

After consolidating all cart logic into a single `CartContext.jsx` file, the ESLint React fast-refresh plugin rejected the file because it exported both a context provider component and a custom hook from the same module.

Solution: The hook (`useCart.js`) and the context value builder (`cartContextValue.js`) were extracted into dedicated files. The provider file was left with only the provider component export, which satisfied the lint rule and also improved separation of concerns.

6.3 ETH-to-Wei conversion precision loss

An early implementation of the payment amount conversion used standard floating-point arithmetic (`amount * 1e18`), which produced rounding errors for prices such as `0.015 ETH` due to IEEE 754 float representation.

Solution: The calculation was rewritten using JavaScript `BigInt`. The ETH string is split at the decimal point, and each part is scaled independently before being summed into a single integer Wei value, eliminating float imprecision entirely.

6.4 React hook warning in wallet detection effect

The initial MetaMask availability check set component state synchronously inside a `useEffect` body, triggering a `set-state-in-effect` lint warning that indicated a potential re-render cycle.

Solution: The availability check was moved into the `useState` initialiser function so it executes once at component creation, before the first render, rather than inside an effect.

6.5 Discogs API rate limiting in development

During album seeding and repeated testing of the Quick Import feature, unauthenticated requests to the Discogs API consistently returned `429 Too Many Requests` after a small number of calls.

Solution: A `DISCOGS_TOKEN` environment variable was added. The server attaches it as an `Authorization: Discogs token=…` header on all Discogs requests, raising the per-minute rate limit from the anonymous threshold to 60 requests per minute.

6.6 Anonymous orders and JWT auth boundary

The checkout flow allows both registered and guest users to complete a purchase to minimise friction. However, the `POST /api/orders` endpoint needed to optionally associate an order with a user ID without failing when no authentication token was present.

Solution: A lenient middleware was created that attempts to decode the JWT if a `Bearer` token is present in the request, attaches `req.user` if decoding succeeds, and calls `next()` regardless of the outcome. The order route then sets `userId: req.user?._id ?? null`, and the `Order` schema marks the field as optional.

7. Project Evolution

7.1 UD1A — Initial Setup

The project started with a React frontend displaying a static album catalog backed by hardcoded mock data. An Express server skeleton was in place but had no database connection. The design identity was established at this stage: a retro/vintage aesthetic with a cream background, dark brown text, burnt orange accents, and Playfair Display typography.

Key deliverables: project structure, working Express server, React app with mock catalog, initial README, design system definition.

7.2 UD1B — Backend Integration and Cart

UD1B replaced mocks with real backend data. MongoDB was connected through a dedicated configuration module and an `Album` Mongoose schema was defined. Two API endpoints were implemented (`GET /api/albums` and `GET /api/albums/:id`). The frontend was refactored to fetch from the API, removing all static data. An album detail page was added with client-side routing via React Router. A shopping cart was built using React Context with `localStorage` persistence.

The folder structure was reorganised from a flat layout into `pages/`, `components/`, `api/`, and `services/` to establish a separation of concerns that would scale through the rest of the project.

Key deliverables: MongoDB connection, album REST API, frontend API integration, album detail page, cart with localStorage persistence, folder restructure.

7.3 UD2A — Wallet UX, Search, and Error Contracts

UD2A added the first visible Web3 feature: a MetaMask wallet connect flow in the header that shows the user's shortened wallet address once connected. Cart behaviour was deepened with increment and decrement controls, stock-capped quantities, per-line subtotals, and visible warnings when stock limits are reached. The catalog gained

real-time debounced search, genre filtering, and multi-option sorting.

API error handling was hardened: the backend now explicitly distinguishes between malformed IDs (HTTP 400) and valid but non-existent IDs (HTTP 404), and the frontend surfaces the actual error message from the response body instead of a generic fallback.

Two automated check scripts were added: a backend API contract verifier and a frontend cart state transition checker.

Key deliverables: MetaMask connect, enriched cart controls, catalog search/filter/sort, API error contracts, test scripts.

7.4 UD3 — Complete Application

UD3 completed the remaining planned items and introduced several unplanned features that raised the project to a production-ready level. The major additions were:

- **Payment flow:** full `eth_sendTransaction` integration on Sepolia, with BigInt-safe ETH□Wei conversion, TX hash display, and Etherscan deep links.
- **Authentication:** user registration and login with JWT, protected route guards, profile management, password change, and saved delivery address.
- **Orders:** order creation on payment success, automatic stock decrementation, user-facing order history.
- **Email:** Resend integration for post-payment receipt emails.
- **Admin panel:** full album CRUD, batch operations, Discogs Quick Import, iTunes audio fill, CSV export, order management with TX hash search and date filtering, live dashboard statistics.
- **Wishlist:** toggle from catalog and detail page, dedicated page, header badge, localStorage persistence.
- **Enriched catalog:** featured album section, special offers, similar albums, recently viewed shelf, skeleton loading cards, stock status badges.
- **Audio player:** custom HTML5 player with seek bar, time display, and buffering indicator.
- **PWA:** web app manifest and custom icon.

8. Conclusions and Future Improvements

8.1 What Was Achieved

VinylEth grew from a static mock catalog into a complete, multi-role web application across four development phases. The final product covers the full customer journey from catalog browsing to cryptocurrency payment confirmation, includes a production-grade admin panel, and integrates three external APIs (Discogs, iTunes, Resend). A total of 111 discrete features were implemented and verified through a combination of automated scripts and manual testing.

The project demonstrates several technically non-trivial integrations working together in a single coherent application: stateless JWT authentication, BigInt-safe ETH payment conversion, Discogs metadata mapping with tracklist parsing, iTunes audio preview fetching, and transactional email delivery. The codebase is organised clearly enough to be understood and extended by a developer unfamiliar with the project.

8.2 Possible Future Improvements

Smart contract escrow. The current payment sends ETH directly to a static wallet address. A real deployment would use a Solidity escrow contract that holds funds until order fulfilment, providing buyer protection and enabling programmatic refunds.

Reservation during checkout. Stock is decremented at order creation, but there is no reservation mechanism during the checkout flow itself. Two users completing checkout simultaneously could both succeed even if only one unit remains. A short-lived reservation at cart checkout entry would solve this.

Image uploads. Album cover images are currently entered as URLs (including those returned by Discogs). A file upload feature with server-side storage — for example via Cloudinary — would be more practical for day-to-day store management.

Automated test suite. The existing check scripts are lightweight contract verifiers. A proper test suite with Vitest on the frontend and Jest combined with Supertest on the backend would provide regression coverage as the application grows.

Cursor-based pagination. The current “Load more” approach appends results to a growing list. Server-side cursor-based pagination would be more efficient at scale and would also enable back-navigation to a specific page.

Unified auth roles. Admin access is currently a separate password-based system independent of user accounts. Integrating it as a role flag on the `User` model would unify the authentication system and allow granting or revoking admin access per account without changing a shared password.